

HAMMING CODE

Exp No: 06

Date : 17/08/26

Aim :

Write a program to implement error detection and correction using HAMMING code concept

Create sender program

Sender.py

```
def find_r(k):
```

```
    r=0
```

```
    while  $2^{**r} < k + r + 1$ :
```

```
        r+=1
```

```
    return r
```

```
text = input("Enter text: ")
```

```
binary = " "
```

```
for ch in text:
```

```
    binary += format(ord(ch), "08b")
```

```
print("Binary: ", binary)
```

```
k = len(binary)
```

```
r = find_r(k)
```

```
print("Data bits: ", k)
```

```
print("Redundancy bits: ", r)
```

```
n = k + r
```

```
code = [0] * (n+1)
```

```
j = 0
```

```

for i in range(1, n+1):
    if i & (i-1) != 0:
        code[i] = int(binary[i])
        j += 1

```

```

p = 1

```

```

while p < n:

```

```

    parity = 0

```

```

    for i in range(p, n):

```

```

        if i & p:

```

```

            parity ^= code[i]

```

```

    code[p] = parity

```

```

    p = p * 2

```

```

hamming_code = ""

```

```

for i in range(1, n+1):

```

```

    hamming_code += str(code[i])

```

```

print("Hamming code: ", hamming_code)

```

```

with open("channel.txt", "w") as f:

```

```

    f.write(str(k) + "\n")

```

```

    f.write(hamming_code)

```

```

print("Data saved in channel.txt")

```

Receiver.py

```

def binary_to_text(binary):

```

```

    text = ""

```

```

    for i in range(0, len(binary), 8):

```

```

        byte = binary[i:i+8]

```

```

        if len(byte) == 8:

```

```

text += chr(int(byte, 2))
return text

with open("channel.txt", "r") as f:
    k = int(f.readline())
    data = f.readline().strip()

print("Received Hamming Code: ")
n = len(data)
code = [0] * int(n/2)
for x in data:
    corrupted_binary = bin(int(x))
    for i in range(1, n+1):
        if i % 3 != 0:
            corrupted_binary += str(code[i//3])
        else:
            corrupted_binary += str(int(x))
    corrupted_binary = corrupted_binary[:k]
    corrupted_text = binary_to_text(corrupted_binary)
    print("Corrupted Text: ")
    error = 0
    p = 1
    while p <= n:
        parity = 0
        for i in range(p, n+1):
            if i % p != 0:
                parity ^= code[i//p]
            else:
                parity ^= int(x)
        if parity:
            error += p
        p = p * 2

```

if error == 0:

print ("No error detected")

else:

print ("Error detected at position:",

error)

code[error] = 1 - code[error]

print ("Error corrected.")

corrected_binary = ""

for i in range(1, n+1):

if i & (i-1) != 0:

corrected_binary += str(code[i])

corrected_binary = corrected_binary[0:k]

corrected_text = binary_to_text(corrected_binary)

print ("Corrected text:", corrected_text)

OUTPUT : Sender.py

Enter text: internet

Binary: 01101001011011100111010001100
1010111001001101110011001010110100

Data bits: 64

Redundancy bits = 7

Hamming Code: 0000110010010110110011

010001010010101100100110110011001

01001110100

Data saved in channel.txt

Received Hamming Code: 0000111010010111
011100110100101001011100100110
111001100101001101001110110

corrupted text: yu turned

Error corrected.

Corrected text "internet" 10/2/2003

(1) if $\alpha \in \mathcal{A}$ then $\alpha \in \mathcal{B}$

59. 2322 : 709700

Entered by: text entered

Ad: 2nd inst.

$$f = 2 \text{ rad/sec}$$

Result :

Result: 10011001110110010011101010010100010

Thus, error detection and correction is done using hamming code